



南開大學
Nankai University

计算机学院
数据安全实验报告

实验 3: 秘密共享实践

姓名：王旭

学号：2312166

专业：计算机科学与技术

2026 年 5 月 19 日

目录

1 实验名称	2
2 实验目的	2
3 实验原理	2
3.1 秘密共享的基础概念与原理	2
3.2 门限秘密共享 (Threshold Secret Sharing) 详细介绍	2
3.3 Shamir 秘密共享核心机制	3
3.4 拉格朗日插值原理与重构	3
3.5 Shamir 机制的加法同态特性	3
3.6 JSON 数据格式的介绍与应用	4
4 实验环境配置流程	4
4.1 实验代码架构设计	4
4.1.1 两个代码各自的作用	4
4.1.2 两个代码之间的联系与协同	5
4.2 实际配置与开发中遇到的问题与解决方法	5
5 实验进行流程	6
5.1 实现流程与思路	6
5.2 具体实现代码	6
5.2.1 底层数学逻辑模块 <code>shamir_math.py</code>	6
5.2.2 主业务与通信逻辑调度脚本 <code>main.py</code>	8
5.3 代码运行结果与分析	9
6 实验总结	11
6.1 主要工作	11
6.2 实验心得	11
7 github 仓库	11

1 实验名称

秘密共享实践

2 实验目的

实验 3.2 假设有三个同学需要对班里的优秀干部 Alice、Bob、Charles、Douglas 进行投票，最后统计各个班干部获得的票数。这个时候就可以利用 Shamir 秘密共享将各个投票方的投票分享出去并进行隐私求和计算。

借鉴实验 3.2，实现三个人对于他们拥有的数据的平均值的计算。

3 实验原理

3.1 秘密共享的基础概念与原理

秘密共享 (Secret Sharing) 是一种将秘密分割并存储的密码技术，目的是阻止秘密过于集中，以达到分散风险和容忍入侵的目的。在传统的加密场景中，如果密钥或隐私数据保存在单个节点上，一旦该节点遭受攻击或发生故障，就会导致数据的整体泄露或永久丢失。

秘密共享通过特定的数学算法，将一个完整的秘密数据分割成多个“碎片 (Shares)”，并将这些碎片分发给不同的参与者或者存储节点。在这种机制下，任何单独的参与者都无法通过自己手中的那一份碎片还原出原始的秘密。只有当参与者将碎片集中起来，并且凑够预先设定好的数量时，才能通过相应的逆向数学计算将原始秘密还原。目前，秘密共享已成为多方安全计算 (MPC)、门限密码学、拜占庭容错系统等领域的核心底层密码学工具。

3.2 门限秘密共享 (Threshold Secret Sharing) 详细介绍

在多方参与的环境下，并不是必须凑齐“所有”的碎片才能还原秘密。如果规定必须全员参与才能解密，那么一旦有某一个节点宕机离线，整个系统的数据就会陷入死锁不可用状态。为了解决这个问题，密码学中引入了“门限 (Threshold)”的概念，也就是 (k, n) 门限秘密共享体制。

- **参数 n** : 表示系统总共有 n 个参与方 (即秘密被分成了 n 个碎片)。
- **参数 k** : 表示重构原始秘密所至少需要的碎片数量 (即门限值/阈值)。

该体制必须满足两个非常严苛的安全数学特性:

1. **可重构性**: 在 n 个碎片中，任意收集到大于或等于 k 个碎片，就可以完美且唯一地复原出原始的秘密。
2. **完美安全性 (Perfect Secrecy)**: 如果收集到的碎片数量小于 k (即使收集了 $k-1$ 个)，那么对于重构原始秘密不仅是“很难计算”，而且是数学上完全提供不了任何关于原始秘密的信息。攻击者拥有 $k-1$ 个碎片去猜测秘密的成功率，与拥有 0 个碎片去盲猜的成功率完全等同。

3.3 Shamir 秘密共享核心机制

Shamir 秘密共享 (Shamir's Secret Sharing, 简称 SSS) 是由著名密码学家 Adi Shamir 于 1979 年提出的一种经典的 (k, n) 门限共享方案。它的基础是代数中的**多项式插值与解析几何定理**：“在二维平面上，给定 k 个点，可以唯一确定一个最高次为 $k - 1$ 次的多项式”。

具体流程如下：

1. **构造有限域**：所有的计算并非在普通的实数轴上进行，而是转移到了模一个大素数 P 的有限域 (Galois Field) 上进行操作。这是因为有限域可以防止计算溢出，并且基于模运算特性保证 $k - 1$ 份碎片不会因为“数字大小的范围暗示”而泄露秘密的线索。
2. **生成多项式**：假设我们需要隐藏的秘密为 S 。我们构造一个 $k - 1$ 次的随机多项式 $f(x) = a_0 + a_1x^1 + a_2x^2 + \dots + a_{k-1}x^{k-1} \pmod{P}$ 。
 - 其中常数项 a_0 就设定为我们隐藏的秘密本身，即 $a_0 = S$ 。
 - a_1 到 a_{k-1} 则是临时产生的完全随机的整数。
3. **切分与分发**：为了给 n 个参与者分发秘密，我们从 $x = 1, 2, 3, \dots, n$ 依次带入多项式计算出 y 值。生成的每一个坐标对 (x_i, y_i) 就是分发给第 i 个人的碎片。

3.4 拉格朗日插值原理与重构

当需要恢复原始秘密时（根据门限规定，收集到了随便 k 份碎片），我们需要从这些 (x, y) 坐标点点集中找出多项式并求得常数项 a_0 （因为 $S = f(0)$ ）。这个操作在数学上通过**拉格朗日插值法 (Lagrange Interpolation)** 实现。

对于收集到的任意 k 个坐标点 (x_i, y_i) ，多项式在 $x = 0$ 处的值可以通过以下插值公式直接算出：

$$S = f(0) = \sum_{i=1}^k y_i \prod_{j \neq i, j=1}^k \frac{-x_j}{x_i - x_j} \pmod{P}$$

在计算机代码实现中，分子分母的除法被转换为模 P 下的乘法逆元运算。通过费马小定理 ($A^{-1} \equiv A^{P-2} \pmod{P}$) 配合 Python 快速幂机制，能够以极高的执行效率瞬间完成还原，且运算绝对严密无精度损耗。

3.5 Shamir 机制的加法同态特性

本次“求多方数据平均值”实验的核心在于，Shamir 秘密共享具备**加法同态 (Additive Homomorphic)** 的神奇性质。所谓加法同态，是指“对密文（碎片）直接进行相加，其结果解密后，恰好等于对原始明文直接相加”。

- 假设由参与者 A、B、C 各自分享了秘密 S_a, S_b, S_c ，对应的多项式分别为 $f_a(x), f_b(x), f_c(x)$ 。
- 参与者 1 手里会分别握有这三个多项式在 $x = 1$ 处的取值。此时该参与者直接将三个碎片在本地相加计算： $F(1) = f_a(1) + f_b(1) + f_c(1)$ 。
- 在代数上，多个同次多项式相加，依然是一个同次多项式 $F(x) = f_a(x) + f_b(x) + f_c(x)$ 。而这个新多项式 $F(x)$ 的常数项，正好就是 $S_a + S_b + S_c$ 。

基于这种同态叠加，参与者无需解密和暴露自己的隐私选票或原始数字，只要把各自手里的碎片数值在纵向（同 x 坐标）上累加，随后再交给汇算员通过拉格朗日重构这个叠加后的多项式截距，自然就得到了数据的总和。得到总和之后直接除以人数，便安全地算出了数据的平均值。

3.6 JSON 数据格式的介绍与应用

在此前的一些实现中，多方数据传递利用了大量的、散乱的 `.txt` 文本文件。为了更贴合现实系统的数据传递场景，我们在架构上引入了 JSON。

JSON (JavaScript Object Notation) 是一种轻量级的数据交换格式。它主要采用完全独立于编程语言的文本格式来存储和表示数据。相比之下它具有结构清晰、易于人阅读和编写、并且容易被机器解析等多个优点。

在本次实验中，我们使用 JSON 代替散碎文本文件，将各个通信节点的传递内容结构化。我们构造了一个类似于信箱 (Mailbox) 的 JSON 字典对象。在这个对象体系中，主键即接收方（同学）的 ID，附带的列表则包含了从他处发送过来的碎片数组。如此不仅极大地精简了实验过程中项目目录的 IO 文件混乱问题，也以规范的协议体模拟呈现了现代网络框架中节点间的数据包 (Payload) 流转行为。

4 实验环境配置流程

本次实验的代码均基于 Python 3 标准库完成，无需安装复杂的外部第三方依赖（如 `phe` 或 `cryptography`），主要使用了 Python 内置的 `random`（用于生成多项式随机系数）、`json`（用于网络通信模拟）以及 `os` 库。实验环境的配置非常轻量且易于复现。

4.1 实验代码架构设计

在早期的实验方案中，多方数据传递往往混合在一个大型脚本中，或者通过生成大量零碎的 `.txt` 文本文件来进行流转。为了提高系统的工程规范性，我们将本实验的代码架构严格解耦，拆分为两个功能明确的 Python 脚本，分别存放在 `src` 目录下。

4.1.1 两个代码各自的作用

- `shamir_math.py` :

该文件专门用于抽象和封装 Shamir 秘密共享的核心密码学与数学逻辑。其主要包含两个核心函数：`generate_shares`（负责随机产生多项式系数并生成指定门限碎片的函数）和 `reconstruct_secret`（负责通过拉格朗日插值与模逆运算还原常数项秘密的函数）。该文件不涉及任何具体的业务和通信逻辑，纯粹作为数学工具库存在。

- `main.py` :

该文件是本次实验程序的主入口，负责模拟整个安全多方计算的分布式流程。其主要作用包括：定义参与者和门限参数、初始化真实秘密数据、调用数学库生成对应的全套碎片、通过 JSON 文件模拟生成网络数据包完成分发、模拟各参与节点在本地进行“同态相加”，并最终在汇算节点完成数据的统计、重构与平均值计算。

4.1.2 两个代码之间的联系与协同

这两个代码在设计上是一种“底层提供接口，上层调用服务”的解耦联系。`main.py` 通过 `import` 引用 `shamir_math.py` 中的数学操作能力。借助这种分离设计，即使未来我们将门限值 (k, n) 扩展到了千人甚至万人级别，或者更换底层的有限域素数 P ，也只需要修改底层库，而不会影响上层复杂的通信调度流程，极大地提高了代码的可读性和可维护性。

4.2 实际配置与开发中遇到的问题与解决方法

在本次实验我们的本地配置与代码优化迭代过程中，我们主要遇到了以下两个问题，并进行了针对性的解决：

1. 问题一：复杂低效的本地 I/O 文件（通信流转问题）

- **问题描述：**我们在最开始的基础参考设计中，为了模拟三个学生互相交换碎片，采用了依次创建、读写并保存 9 个单独局部 `.txt` 文件的做法。当参与方扩大时，本地工作区会产生大量的文本文件，这不仅难以核查数据传递的过程，也容易发生文件读取越界或未关闭的异常。
- **解决方法：**我们摒弃了原始的 `txt` 离散文本的读取方式，这种方式较为低效且繁琐，设计并引入了 `communication_simulate.json` 架构。在内部逻辑上构建了一个结构化的“信箱 (Mailbox)”对象，各方交互的全部密文流转状态被统一打包，写入一个整洁的 JSON 映射文件中。这既贴近了现代 Web 服务器中的 JSON 请求载荷 (Payload) 传输体量，又优化了本地运行环境。

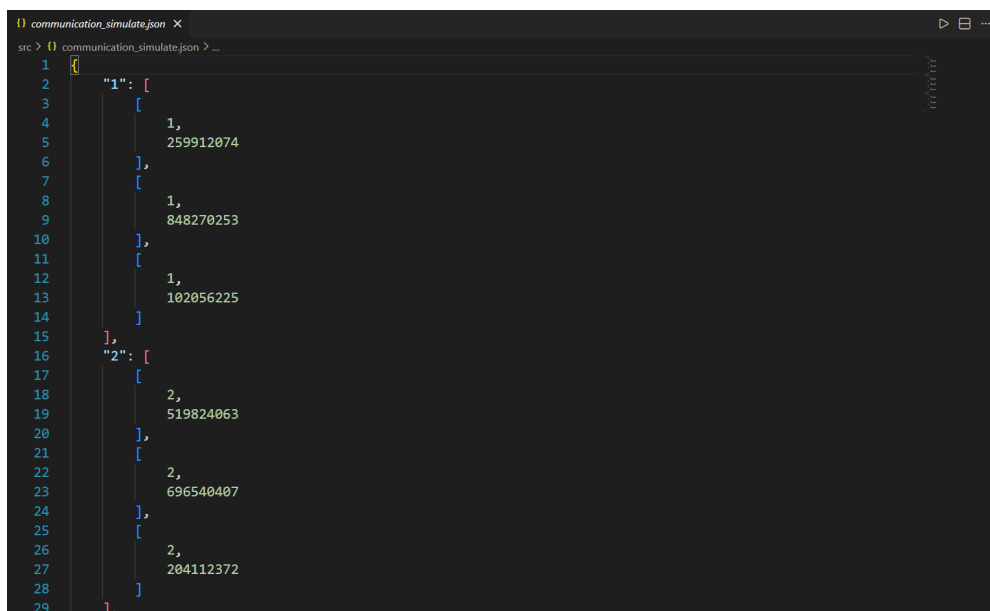


图 4.1: 后续实验代码所生成的 `communication_simulate.json` 文件

2. 问题二：模逆元运算的低效代码

- **问题描述：**在拉格朗日重构时，涉及到有限域内的大素数除法，必须借助费马小定理并配合快速幂算法来求乘法逆元。传统的做法是手写 `quickpower()` 的显式 `while` 循环位运算，

虽然逻辑可行，但在 Python 原生环境下，这种自行封装的基础循环性能较低，并且加大了意外 Bug 出现的风险。

- **解决方法**：我们知道 Python 内置的 `pow(base, exp, mod)` 方法由于是在底层使用 C 语言直接进行优化调用的，性能要远远优于手写的快速幂循环。因此，我们重构了逆元求值部分，使用 `pow(denominator, PRIME - 2, PRIME)` 替换掉了冗长的求模逆元循环。这大幅精简了冗余代码，并提高了系统的执行效率。

5 实验进行流程

5.1 实现流程与思路

本次隐私平均值计算实验遵循以下五个核心阶段进行：

1. **数据初始化阶段**：设定参与门限为 (2,3)，即总共 3 个人参与，凑齐 2 个碎片即可解密。接着随机设定三个同学各自想要隐藏的原始数据（即他们的数字选票或隐私数据）。
2. **生成与切分阶段**：各参与者作为一个独立节点，将自己的秘密数据作为常数（截距）构造随机的一元一次多项式，并沿 x 轴散列取出 3 个点作为秘密碎片（由于都在相同的有限域 P 内，所以坐标点都在巨大的数值空间中）。
3. **分布式分发阶段**：利用统一构建的 `communication_simulate.json` 数据包，模拟各方在网络中把带有自己坐标标识的碎片发送给指定同学。在这个过程中，信道上的数据全是看似随机的碎片，拦截者无法探知原始数字。
4. **本地加法同态聚合**：各同学利用 Shamir 的同态性质，将自己收信箱内坐标位置相同（即 x 均为该同学自身 ID）的所有 y 值相加并取模。此时各同学本地持有的碎片便自然变成了“记录总和秘密的多项式”的其中一张碎片。
5. **集中重构与平均计算**：由独立的汇总终端（或任意两名同学结对）合并他们聚合后的计算结果碎片，利用拉格朗日公式逆向推导出 $x = 0$ 处的值，也就是三人数据的加和。最后对总和除以 3 得到最终被求取的平均值。

5.2 具体实现代码

5.2.1 底层数学逻辑模块 `shamir_math.py`

该文件提供基础的秘密生成与多项式插值重构计算，属于底层运算引擎。

shamir_math.py

```
1 import random
2
3 # 使用经典的大素数，保证有限域安全性且防止溢出
4 PRIME = 1000000007
5
6 def generate_shares(secret, n, k):
7     """
8     根据 Shamir (k, n) 门限方案生成 n 个分享碎片。
```

```

9      """
10     # 随机生成多项式的系数，常数项为 secret
11     coefficients = [secret] + [random.randint(1, PRIME - 1) for _ in range(k - 1)]
12
13     shares = []
14     for x in range(1, n + 1):
15         # 计算多项式在 x 处的值 y = (c0 + c1*x + c2*x^2 + ...) mod PRIME
16         y = sum(c * pow(x, i, PRIME) for i, c in enumerate(coefficients)) % PRIME
17         shares.append((x, y))
18
19     return shares
20
21 def reconstruct_secret(shares):
22     """
23     使用拉格朗日插值法从 k 个碎片中重构秘密。
24     shares: 包含 (x, y) 元组的列表
25     """
26     secret = 0
27     for i, (x_i, y_i) in enumerate(shares):
28         numerator = 1
29         denominator = 1
30         for j, (x_j, _) in enumerate(shares):
31             if i != j:
32                 numerator = (numerator * (0 - x_j)) % PRIME
33                 denominator = (denominator * (x_i - x_j)) % PRIME
34
35         # 使用 Python 内置的 pow 函数和费马小定理快速求逆元，高效直接
36         lagrange_coeff = (numerator * pow(denominator, PRIME - 2, PRIME)) % PRIME
37         secret = (secret + y_i * lagrange_coeff) % PRIME
38
39     return secret

```

代码详细解析：

- **PRIME 变量**：定义了极其巨大的素数 ($10^9 + 7$)，这确保了所有的运算都落在该素数构造的有限域 (Galois Field) 内，既保证了运算的安全性和抗穷举攻击能力，又有效防止了不断乘积与叠加积压可能导致的数据内存溢出。
- **generate_shares 函数**：该函数是秘密分享行为的数学入口。首先通过 `random.randint` 在有限域内随机生成 $k - 1$ 个系数，加上我们的隐秘原始数据 (即常数项 `secret`)，共同构建了一元高次随机多项式。随后通过遍历带入不同的 x 坐标独立计算相关的 y 值，生成了 n 份带有特定点位标识的随机秘密碎片。
- **reconstruct_secret 函数**：该函数承担最终“密码还原”的功能。系统通过传入已凑齐门限数量的聚合碎片列表，严格遵循拉格朗日插值法的数学推导公式推算常数截距。对于数学公式中最易出错的“除分数”运算，这段代码通过费马小定理 ($A^{p-2} \equiv A^{-1} \pmod{p}$) 结合 Python 内建的高层 C 级优化的 `pow()` 运算，一步实现了模逆元的低延时求解。

5.2.2 主业务与通信逻辑调度脚本 main.py

该模块导入了数学引擎，专注于模拟三个参与者的数据网络交互和结果推导。

main.py

```
1 import json
2 import os
3 from shamir_math import generate_shares, reconstruct_secret, PRIME
4
5 def save_json(filename, data):
6     with open(filename, 'w', encoding='utf-8') as f:
7         json.dump(data, f, ensure_ascii=False, indent=4)
8
9 def load_json(filename):
10     with open(filename, 'r', encoding='utf-8') as f:
11         return json.load(f)
12
13 def main():
14     # 定义参与者和门限参数
15     N = 3 # 参与方数量
16     K = 2 # 门限值
17
18     # 1. 初始化数据：三个同学各自的秘密数据
19     secrets = {1: 85, 2: 92, 3: 78}
20     print(f"[阶段 1: 数据产生] 学生的真实数据: {secrets}")
21     print(f"真实的平均值应为: {sum(secrets.values()) / N:.2f}\n")
22
23     # 2. 生成并分发碎片（模拟网络信箱）
24     mailbox = {1: [], 2: [], 3: []}
25
26     for student_id, secret in secrets.items():
27         shares = generate_shares(secret, n=N, k=K)
28         print(f"同学 {student_id} 生成了 3 份碎片: {shares}")
29         # 第 i 份碎片发给同学 i
30         for i in range(1, N + 1):
31             mailbox[i].append(shares[i - 1])
32
33     print("\n[阶段 2: 碎片分发] 正在模拟碎片在网络中的交互传输...")
34     json_path = os.path.join(os.path.dirname(__file__), 'communication_simulate.json')
35     save_json(json_path, mailbox)
36     print(f"碎片的结构化数据包已生成并保存至 {json_path}")
37
38     # 3. 隐私求和：本地读取并加和自己受到的碎片
39     received_mailbox = load_json(json_path)
40     aggregate_shares = []
41
42     print("\n[阶段 3: 本地计算]
43         同学各自对收到了网络发来的碎片，并在本地进行同态相加...")
44     for student_id_str, shares in received_mailbox.items():
```

```
44     x_val = int(student_id_str)
45     # 将接收到的所有 y 相加并求模
46     y_sum = sum(share[1] for share in shares) % PRIME
47     final_share = (x_val, y_sum)
48     aggregate_shares.append(final_share)
49     print(f"同学 {student_id_str} 本地计算完成, 聚合碎片 d{student_id_str} 为:
50           {final_share}")
51
52     # 4. 统计与重构: 获取大于等于 K 份的结果还原多项式
53     chosen_shares = aggregate_shares[:K]
54     print(f"\n[阶段 4: 汇总重构] 汇算节点收集了 {K} 份结果用于重构秘密总和:
55           {chosen_shares}")
56
57     total_sum = reconstruct_secret(chosen_shares)
58     print(f"-> 【成功】 重构出数据的求和总值为: {total_sum}")
59
60     # 5. 求平均
61     average = total_sum / N
62     print(f"-> 【完成】 最终计算得出 3 人的数据平均值为: {average:.2f}")
63
64 if __name__ == "__main__":
65     main()
```

代码详细解析:

- **数据定义与信道封装:** `secrets` 字典模拟了三个用户的初始隐私输入数据。`mailbox` 及关联的 `save_json` / `load_json` 读写模块, 取代了早期低效实验中建立大量临时 `.txt` 的繁杂动作。这套机制能够将各密文节点待发送与接收到的碎片以高可读的标准 JSON 键值对形式进行存储和解译, 还原出了真实分布式物理架构下的异步传值与 Payload 负载数据分发流程。
- **同态叠加与协同盲计算 (阶段三):** 在阶段三的遍历循环中, 由于所有用户发送给目标学生 i 的碎片截段, 其横坐标 x 值必然是相同的 (均为 i), 因此这部分代码利用了 Shamir 多项式的加法同态特性。我们直接采用了 `sum(share[1] ... ) % PRIME` 的编程范式, 对收到所有的 y 坐标做模余相加, 各计算方在未知原始数值的状态下, 参与完成了密文叠加态的共同构建。
- **结果推导与平均计算 (阶段四与五):** 最后 `chosen_shares` 按照预设的门限参数提取了前 K 个多方聚合后的复合碎片并投入 `reconstruct_secret` 计算。在得出所有人的数据加和 `total_sum` 后, 除以参与者数量 N 即可满足“计算三个人数据的平均值”的具体要求。

5.3 代码运行结果与分析

我们在开发环境终端中直接运行 `python main.py`, 得到如下标准输出:

```

PS D:\数据安全\实验三\src> python main.py
[阶段 1: 数据产生] 学生的真实数据: {1: 85, 2: 92, 3: 78}
真实的平均值应为: 85.00

同学 1 生成了 3 份碎片: [(1, 259912074), (2, 519824063), (3, 779736052)]
同学 2 生成了 3 份碎片: [(1, 848270253), (2, 696540407), (3, 544810561)]
同学 3 生成了 3 份碎片: [(1, 102056225), (2, 204112372), (3, 306168519)]

[阶段 2: 碎片分发] 正在模拟碎片在网络中的交互传输...
碎片的结构化数据包已生成并保存至 communication_simulate.json

[阶段 3: 本地计算] 同学各自对收到了网络发来的碎片, 并在本地进行同态相加...
同学 1 本地计算完成, 得出的聚合碎片 d1 为: (1, 210238545)
同学 2 本地计算完成, 得出的聚合碎片 d2 为: (2, 420476835)
同学 3 本地计算完成, 得出的聚合碎片 d3 为: (3, 630715125)

[阶段 4: 汇总重构] 汇算节点收集了 2 份结果用于重构秘密总和: [(1, 210238545), (2, 420476835)]
-> 【成功】重构出数据的求和总值为: 255
-> 【完成】最终计算得出 3 人的数据平均值为: 85.00
PS D:\数据安全\实验三\src>

```

图 5.2: 终端运行结果

```

1 PS D:\数据安全\实验三\src> python main.py
2 [阶段 1: 数据产生] 学生的真实数据: {1: 85, 2: 92, 3: 78}
3 真实的平均值应为: 85.00
4
5 同学 1 生成了 3 份碎片: [(1, 259912074), (2, 519824063), (3, 779736052)]
6 同学 2 生成了 3 份碎片: [(1, 848270253), (2, 696540407), (3, 544810561)]
7 同学 3 生成了 3 份碎片: [(1, 102056225), (2, 204112372), (3, 306168519)]
8
9 [阶段 2: 碎片分发] 正在模拟碎片在网络中的交互传输...
10 碎片的结构化数据包已生成并保存至 communication_simulate.json
11
12 [阶段 3: 本地计算] 同学各自对收到了网络发来的碎片, 并在本地进行同态相加...
13 同学 1 本地计算完成, 得出的聚合碎片 d1 为: (1, 210238545)
14 同学 2 本地计算完成, 得出的聚合碎片 d2 为: (2, 420476835)
15 同学 3 本地计算完成, 得出的聚合碎片 d3 为: (3, 630715125)
16
17 [阶段 4: 汇总重构] 汇算节点收集了 2 份结果用于重构秘密总和: [(1, 210238545), (2,
18 420476835)]
19 -> 【成功】重构出数据的求和总值为: 255
20 -> 【完成】最终计算得出 3 人的数据平均值为: 85.00
PS D:\数据安全\实验三\src>

```

运行结果与指标分析:

1. 真实数据和最终结果的精准校对

我们在第一阶段设置的各方预设数字是 {1: 85, 2: 92, 3: 78}, 通过直接相加可以求得明文真实平均值是 85.00。与我们在最终阶段解密总和 255 进而除以人数 3 得到的平均值 85.00 一致, 证实了底层拉格朗日数学计算精度极高、逻辑流程无误。

2. 秘密切分的安全性分析 (阶段一)

以同学 1 的秘密值 85 为例, 分给不同节点的碎片 y 值高达 259912074 等随机数, 与目标值相差很大。这也体现了有限域 $P = 1000000007$ 的数值掩盖作用。

3. 加法同态混淆性与正确性 (阶段三)

当同学各自收到碎片后进行求和时 (比如同学 1 得出的 d_1 : 210238545), 由于依然处于密文状

态的多项式叠加层中，各个运算者其实对别人的原本分数依然一无所知；在这个过程中我们顺利完成了无暴露下的密文数据的聚合。

4. 门限机制灵活性（阶段四）

重构时系统任意随机抽取了 $(1, \dots)$ 和 $(2, \dots)$ 这两份子结果。通过这两点坐标构成的唯一多项式重构结果，达到了 255 的求和值。说明在任意节点突然下线（即只剩两个有效响应的统计服务器节点），系统也能计算出结果，体现了 $(2, 3)$ 门限下的系统容错性。

6 实验总结

6.1 主要工作

在本次基于 Shamir 秘密共享的数据隐私计算实验中，我们围绕“三方数据的隐私求和与平均值计算”完成了以下工作：

- 1. 理论研究与代码设计：**深入研究了 (k, n) 门限机制、拉格朗日插值法以及 Shamir 算法的加法同态特性。在代码实现上，将底层密码学核心引擎（`shamir_math.py`）与上层分布式通信调度逻辑（`main.py`）进行了独立封装。
- 2. 通信机制的优化：**摒弃了传统实现中低效、复杂的 `txt` 多文本读写模拟的方式，引入了规范的 JSON 文件数据结构，以类似于现代网络中 Payload 数据包的形式实现了多节点间密文数据的安全传输。
- 3. 完成实验目标：**成功模拟了三名同学生成碎片、网络分发、密文状态下的本地同态求和，并在最终只抽取两个聚合碎片的情况下还原出了原始数据的合计值，完美计算出了平均值，验证了系统在隐私保护下的严密性与高容错性。

6.2 实验心得

通过本次实验，我将我们在《数据安全》课堂上学到的密码学理论（如 Galois 有限域、门限秘密共享、安全多方计算 MPC 等）成功转化为了可执行落地的代码系统。在工程实现层面，本次实验很好地锻炼了我的代码重构与优化能力。对比此前参考方案中大量使用低效、复杂的 `txt` 文本的方式，我在本实验中采用了 JSON 信箱报文的打包收发这一优化手段。这让我认识到：优秀的密码程序不仅需要理论运算上的正确，同时也需要应用架构层面的系统性。

总之，本次实验让我在秘密共享的理论学习、实验代码架构的设计两个方面得到了很好地锻炼，收获很大。

7 github 仓库

代码详见：<https://github.com/Wangxuuuuu/Data-Security/tree/main>